

REST Engine

- [1. REST-Engine – Kompakte System-Referenz für KI-Modelle](#)
- [2. Operationen \(DSL op\)](#)
 - [2.1. EXEC \(Standardoperation für HTTP-Aufrufe\)](#)
 - [2.2. ASSERT \(Validierung der Antwort\)](#)
 - [Erweiterte JSONPath-Evaluierung innerhalb der BODY-Assertion:](#)
 - [2.2.1. ASSERT MATCH_REFERENCE – Validierung kompletter REST-Responses](#)
 - [Grundprinzip](#)
 - [Erster Testlauf – automatische Erstellung der Referenz](#)
 - [Nachfolgende Testläufe – Vergleich mit der Referenz](#)
 - [Semantischer JSON-Vergleich](#)
 - [Dynamische Felder mit ignore ausschließen](#)
 - [Arrays mit nicht garantierter Reihenfolge](#)
 - [Root-Arrays](#)
 - [Kombination von ignore und unordered](#)
 - [Referenzdateien](#)
 - [Vollständiges Beispiel](#)
 - [Wann sollte MATCH_REFERENCE verwendet werden?](#)
 - [Parameterübersicht](#)
 - [2.3. BUFFER \(Extraktion in Variablen\)](#)
- [3. Referenzübersicht der Parameter nach Operationen](#)
- [Praxis-Tutorial: Kompletter REST-Lebenszyklus](#)
 - [Komplettes YAML-Szenario](#)
 - [Darstellung der Konsolenausgabe \(Log-Output\)](#)
- [Troubleshooting & Häufige Fehler](#)

REST Engine ermöglicht die Ausführung von HTTP-Aufrufen (REST API) innerhalb von Testschritten. Der Engine unterstützt das Senden dynamischer HTTP-Anfragen (mit Platzhaltern, aufgelösten Variablen und der vordefinierten Ersetzung von `{{endpoint}}`), einen automatischen Retry-Mechanismus bei instabilen Verbindungen, die Assertion (Validierung) von Antworten über eine erweiterte Jayway-JsonPath-Engine sowie das Speichern von Werten aus der Antwort in einen temporären Speicher-Buffer zur späteren Verwendung im Test.

1. REST-Engine – Kompakte System-Referenz für KI-Modelle

1 | YAML

```
1 # AGATE REST-Engine Syntax Referenz
2 # Für die Generierung von Testschritten und Analysen verwenden
3
4 - type: REST
5   op: EXEC
6   command: rest.countryinfo.ListOfCountryNamesGroupedByContinent # dot.
7   response: "my_response" # Schlüssel, unter dem das RestResponse
8
9 - type: REST
10  op: ASSERT
11  response: "my_response" # Schlüssel der gespeicherten Antwort a
12  source: "STATUS|BODY|HEADERS" # Was validiert wird
13  path: "$.data.id" # JsonPath für BODY, Header-Name für HE
```

```

14  action: "EQUALS|CONTAINS|EXISTS|COUNT|IS_EMPTY" # Typ des Vergleichs
15  expected: "1001"                                # Erwarteter Wert (Skalar, Regex oder V
16  value: "trazeni_tekst"                            # Optional: Wird bei der COUNT-Aktion z
17
18  - type: REST
19  op: BUFFER
20  response: "my_response"                          # Quelle, die gelesen wird
21  source: "STATUS|BODY|HEADERS"
22  path: "$.data.token"                             # Jackson-Pointer-Syntax (wird aus $.pa
23  name: "auth_token"                               # Name der Variable, in der der Wert ge
24

```

2. Operationen (DSL op)

Der REST-Engine erkennt die Operation anhand des Parameters `op`. Wenn der Parameter `op` nicht definiert ist, lautet der Standardwert **EXEC** (Ausführung des HTTP-Aufrufs).

2.1. EXEC (Standardoperation für HTTP-Aufrufe)

Erstellt und führt eine HTTP-Anfrage mit dem integrierten `HttpClient` aus. Die SSL-Verifizierung ist standardmäßig deaktiviert (akzeptiert selbstsignierte Zertifikate) und der Timeout beträgt 10 Sekunden.

- **Endpoint-Ersetzung:** Wenn der Schritt den Parameter `endpoint` enthält, wird der Wert innerhalb von `url`, der `{{endpoint}}` enthält, automatisch durch diesen Wert ersetzt (wobei ein abschließender Schrägstrich am Ende entfernt wird).
- **Logik für spezifische Werte in Headern:**
 - `{NULL}` – Der Header wird komplett übersprungen und nicht in der Anfrage gesendet.
 - `{EMPTY}` – Ein leerer String (`" "`) wird als Wert des Headers übergeben.
- **Integrierter Retry-Mechanismus:** Wenn die Verbindung vom Server unterbrochen wird (`Connection reset` oder `header parser received no bytes`), wartet der Engine automatisch 10 ms und versucht es erneut (maximal 3 Versuche), bevor der Test fehlschlägt.
- **Antwort speichern:** Das Ergebnis des Aufrufs (`RestResponse` , das den Statuscode, den Body und die Header enthält) wird zwingend im Kontext unter dem im Parameter `response` angegebenen Schlüssel gespeichert.

2.2. ASSERT (Validierung der Antwort)

Evaluert die Daten aus dem gespeicherten `RestResponse` -Objekt. Unterstützt die Arbeit mit drei Datenquellen (`source`): `STATUS` , `BODY` und `HEADERS` .

Erweiterte JSONPath-Evaluierung innerhalb der BODY-Assertion:

Zur Überprüfung des Antwort-Bodys wird die Bibliothek **Jayway JsonPath** verwendet.

Unterstützte Aktionen sind:

- **EXISTS:** Prüft, ob der Pfad existiert. Wenn der Pfad ein Array von Elementen zurückgibt (z. B. Wildcard `[*].id`) und der Parameter `expected` übergeben wurde, ist die Aktion

erfolgreich, wenn **mindestens ein** Element im Array dem erwarteten Wert entspricht.

- **COUNT:** Gibt die Anzahl der Elemente im Array zurück. Wenn der Parameter `value` übergeben wird, zählt der Engine nur die Elemente im Array, deren Wert gleich dem Parameter `value` ist.
- **CONTAINS:** Wenn der Pfad ein Array von Elementen zurückgibt, wird geprüft, ob der erwartete String in diesem Array enthalten ist. Wenn ein einzelner Skalar zurückgegeben wird, wird eine standardmäßige `String.contains` -Prüfung durchgeführt.
- **EQUALS / VERIFY / NOT_EQUALS:** Standardmäßige Text- und numerische Vergleiche. Ein Wildcard mit mehreren Ergebnissen ist hier nicht zulässig und führt zu einem Fehler (es muss ein genauer Array-Index angegeben werden, z. B. `[$[0]]`).

2.2.1. ASSERT MATCH_REFERENCE – Validierung kompletter REST-Responses

Neben der gezielten Validierung einzelner Werte über JSONPath unterstützt die REST-Engine mit `MATCH_REFERENCE` auch den **semantischen Vergleich einer vollständigen JSON-Response mit einer gespeicherten Referenzantwort.**

Diese Variante eignet sich insbesondere für Regressionstests, bei denen nicht nur einzelne Felder, sondern die komplette fachliche Struktur einer REST-Antwort überprüft werden soll.

Anstatt für jedes einzelne Feld einen eigenen ASSERT zu definieren, kann die gesamte Response mit einem einzigen Testschritt validiert werden.

Grundprinzip

`MATCH_REFERENCE` wird auf eine zuvor durch einen REST-EXEC-Schritt gespeicherte Response angewendet:

```
1 - type: REST
2   op: EXEC
3   command: rest.jsonplaceholder.typicode.todos.1
4   endpoint: "{B[apiEndpoint]}"
5   response: response_rest1
6
7 - id: verify_todo_response
8   type: REST
9   op: ASSERT
10  response: response_rest1
11  source: BODY
12  action: MATCH_REFERENCE
```

Im Gegensatz zu einer klassischen BODY-Assertion werden für `MATCH_REFERENCE` **kein path und kein expected benötigt.**

AGATE vergleicht den vollständigen JSON-Body mit einer permanent gespeicherten JSON-Referenz.

Erster Testlauf – automatische Erstellung der Referenz

Existiert für den Testschritt noch keine Referenzdatei, speichert AGATE die aktuelle REST-Response automatisch als neue Referenz.

Beispiel:

```
1 >>> REST ASSERT REFERENCE CREATED :
2   .../references/rest_engine_matcher_demo/
3   TC_01__verify_todo_response.json
4
5 >>> REVIEW REQUIRED :
6   Check the generated reference response and keep/commit it only if
   it is correct.
```

Der Test wird in diesem Fall **nicht als fehlgeschlagen bewertet**.

Die erzeugte JSON-Datei muss anschließend einmalig vom Tester fachlich überprüft werden.

Ist die Response korrekt, bleibt die Referenz bestehen und kann gemeinsam mit dem Testfall in Git eingchecked werden.

Wichtig: Eine bereits vorhandene Referenz wird von AGATE nicht automatisch überschrieben. Eine fachlich geprüfte Baseline kann daher nicht versehentlich durch eine spätere, möglicherweise fehlerhafte Response ersetzt werden.

Nachfolgende Testläufe – Vergleich mit der Referenz

Existiert bereits eine Referenzdatei, wird die aktuelle REST-Response mit dieser Referenz verglichen.

Sind beide Responses semantisch identisch, ist der ASSERT erfolgreich:

```
1 >>> REST ASSERT SUCCESS: MATCH_REFERENCE
2   Reference: .../TC_01__verify_todo_response.json
```

Tritt eine fachlich relevante Abweichung auf, schlägt der ASSERT fehl.

Beispiel:

```
1 >>> REST ASSERT MATCH_REFERENCE: FAILED
2 >>> REFERENCE:
3   .../TC_01__verify_todo_response.json
4
5 >>> DIFFERENCE [1] VALUE_CHANGED
6   path      : $.title
7   expected  : delectus aut autem
8   actual    : neuer Titel
```

Damit ist unmittelbar erkennbar:

- an welcher Stelle die Response abweicht,
- welcher Wert in der Referenz erwartet wurde,
- welcher Wert tatsächlich geliefert wurde.

Auch strukturelle Änderungen können erkannt werden, beispielsweise fehlende oder neu hinzugekommene Felder.

Semantischer JSON-Vergleich

`MATCH_REFERENCE` führt keinen einfachen String-Vergleich der beiden JSON-Dokumente durch.

Die Responses werden als JSON-Strukturen interpretiert und semantisch verglichen.

Dadurch führen beispielsweise unterschiedliche Formatierungen nicht zu einem Fehler.

Diese beiden Responses sind somit identisch:

```
1 {  
2   "userId": 1,  
3   "id": 1,  
4   "title": "delectus aut autem",  
5   "completed": false  
6 }
```

und:

```
1 {"completed":false,"title":"delectus aut autem","id":1,"userId":1}
```

Insbesondere sind beim Vergleich irrelevant:

- Whitespaces,
- Zeilenumbrüche,
- Einrückungen,
- Reihenfolge der Properties innerhalb eines JSON-Objekts.

Fachlich relevante Unterschiede werden dagegen erkannt, beispielsweise:

- geänderte Werte,
- fehlende Properties,
- zusätzliche Properties,
- unterschiedliche Datentypen,
- unterschiedliche Array-Inhalte.

Dynamische Felder mit `ignore` ausschließen

REST-Responses enthalten häufig technische Werte, die sich bei jedem Aufruf ändern, beispielsweise:

- Zeitstempel,
- Request-IDs,
- Transaction-IDs,
- UUIDs,
- technische Laufzeitinformationen.

Solche Felder können über `ignore` vom Vergleich ausgeschlossen werden.

Beispiel:

```
1 - id: verify_customer_response
2   type: REST
3   op: ASSERT
4   response: customer_response
5   source: BODY
6   action: MATCH_REFERENCE
7
8   ignore:
9     - "$.timestamp"
10    - "$.requestId"
11    - "$.transactionId"
```

Die angegebenen JSON-Pfade werden sowohl aus der gespeicherten Referenz als auch aus der aktuellen Response für den Vergleich ausgeschlossen.

Alle anderen Felder werden weiterhin vollständig geprüft.

Best Practice: `ignore` sollte ausschließlich für Werte verwendet werden, deren Änderung fachlich keine Bedeutung besitzt. Fachliche Daten sollten nicht ignoriert werden, da dadurch echte Regressionen unentdeckt bleiben könnten.

Arrays mit nicht garantierter Reihenfolge

Bei REST-Schnittstellen ist die Reihenfolge von Elementen innerhalb eines Arrays nicht immer fachlich relevant oder technisch garantiert.

Beispielsweise können folgende Responses fachlich identisch sein:

```
1 {
2   "readers": [
3     {
4       "id": 100,
5       "name": "Reader A"
6     },
7     {
```

```

8     "id": 200,
9     "name": "Reader B"
10  }
11  ]
12  }

```

und:

```

1  {
2    "readers": [
3      {
4        "id": 200,
5        "name": "Reader B"
6      },
7      {
8        "id": 100,
9        "name": "Reader A"
10     }
11   ]
12 }

```

Mit `unordered` kann definiert werden, dass die Reihenfolge der Elemente beim Vergleich keine Rolle spielt:

```

1  - id: verify_readers_response
2    type: REST
3    op: ASSERT
4    response: response_1
5    source: BODY
6    action: MATCH_REFERENCE
7
8    unordered:
9      - path: "$.readers"
10        matchBy: "$.id"

```

`path` definiert das Array, dessen Reihenfolge ignoriert werden soll.

`matchBy` definiert einen stabilen Schlüssel innerhalb der Array-Elemente, anhand dessen die Objekte einander zugeordnet werden.

Im Beispiel werden Reader anhand ihrer `id` miteinander verglichen.

Root-Arrays

Auch eine REST-Response, deren Root-Element direkt ein JSON-Array ist, kann vollständig mit `MATCH_REFERENCE` verglichen werden.

Beispiel:

```

1  [
2    {
3      "id": 1,
4      "title": "First"
5    },
6    {
7      "id": 2,
8      "title": "Second"
9    }

```

Soll bei einem solchen Root-Array die Reihenfolge keine Rolle spielen, kann das Root-Array selbst als `unordered` definiert werden:

```

1 - id: verify_posts_response
2   type: REST
3   op: ASSERT
4   response: posts_response
5   source: BODY
6   action: MATCH_REFERENCE
7
8   unordered:
9     - path: "$"
10      matchBy: "$.id"

```

Damit werden die einzelnen Objekte anhand ihrer `id` zugeordnet und unabhängig von ihrer Position im Array verglichen.

Kombination von `ignore` und `unordered`

Beide Mechanismen können gemeinsam verwendet werden.

Beispiel:

```

1 - id: verify_readers_response
2   type: REST
3   op: ASSERT
4   response: readers_response
5   source: BODY
6   action: MATCH_REFERENCE
7
8   ignore:
9     - "$.timestamp"
10    - "$.requestId"
11
12   unordered:
13     - path: "$.readers"
14      matchBy: "$.id"

```

Damit wird die vollständige REST-Response gegen die Referenz geprüft, wobei:

1. `timestamp` ignoriert wird,
2. `requestId` ignoriert wird,
3. die Reihenfolge der Reader keine Rolle spielt,
4. Reader anhand ihrer `id` einander zugeordnet werden,
5. alle übrigen Inhalte vollständig verglichen werden.

Referenzdateien

Die Referenzdateien werden automatisch relativ zum jeweiligen YAML-Testfall unterhalb eines `references` -Verzeichnisses gespeichert.

Das grundsätzliche Schema lautet:

```
1 <yaml-directory>/
2   └─ references/
3       └─ <yaml-name>/
4           └─ <testcase>__<step-id>.json
```

Beispiel:

```
1 references/
2   └─ rest_engine_matcher_demo/
3       └─ TC_01__verify_todo_response.json
4       └─ TC_02__verify_posts_response.json
```

Die `id` des ASSERT-Schritts sollte deshalb bewusst und stabil gewählt werden:

```
1 - id: verify_todo_response
2   type: REST
3   op: ASSERT
4   response: response_rest1
5   source: BODY
6   action: MATCH_REFERENCE
```

Best Practice: Für `MATCH_REFERENCE` sollte immer eine explizite und sprechende `id` vergeben werden. Dadurch bleibt die Zuordnung zur Referenzdatei stabil, auch wenn davor neue Testschritte eingefügt oder bestehende Schritte verschoben werden.

Die Referenzdateien sind Bestandteil des Tests und sollten gemeinsam mit den YAML-Testfällen versioniert werden.

Vollständiges Beispiel

```
1 testCases:
2
3   - id: TC_01
4     description: Simple REST GET Test
5     stage: '*'
6     priority: HIGH
7
8     variables:
9       apiEndpoint: "https://jsonplaceholder.typicode.com"
10
11     steps:
12
13       # REST Request ausführen
14       - type: REST
15         op: EXEC
16         command: rest.jsonplaceholder.typicode.todos.1
17         endpoint: "{B[apiEndpoint]}"
18         response: response_rest1
19
20       # HTTP Status prüfen
21       - type: REST
22         op: ASSERT
```

```

23     response: response_rest1
24     source: STATUS
25     action: EQUALS
26     expected: 200
27
28     # Komplette JSON-Response gegen Referenz prüfen
29     - id: verify_todo_response
30       type: REST
31       op: ASSERT
32       response: response_rest1
33       source: BODY
34       action: MATCH_REFERENCE

```

Beim ersten Lauf erzeugt AGATE die Referenz.

Bei allen weiteren Läufen wird die komplette JSON-Response automatisch gegen diese Referenz validiert.

Damit kann ein vollständiger REST-Response mit einem einzigen ASSERT als Regressionstest abgesichert werden.

Wann sollte MATCH_REFERENCE verwendet werden?

MATCH_REFERENCE eignet sich besonders, wenn:

- die vollständige Response fachlich relevant ist,
- viele Response-Felder geprüft werden müssten,
- Regressionen in bisher nicht explizit geprüften Feldern erkannt werden sollen,
- eine stabile Baseline für einen REST-Endpoint existiert,
- OpenAPI-generierte Tests mit möglichst wenig manuell definierten ASSERTs erweitert werden sollen.

Ein klassischer JSONPath-ASSERT ist dagegen weiterhin sinnvoll, wenn nur ein einzelner konkreter Wert geprüft werden soll:

```

1  - type: REST
2    op: ASSERT
3    response: response_rest1
4    source: BODY
5    path: "$.title"
6    action: EQUALS
7    expected: "delectus aut autem"

```

Die beiden Varianten schließen sich nicht aus und können innerhalb desselben Testfalls kombiniert werden.

Beispielsweise kann zunächst der HTTP-Status gezielt geprüft und anschließend die vollständige Response gegen die Referenz validiert werden.

Parameter	Beschreibung	Erforderlich
<code>id</code>	Stabiler Bezeichner des ASSERT-Schritts und Bestandteil der Referenzzuordnung.	Empfohlen
<code>type</code>	Muss <code>REST</code> sein.	Ja
<code>op</code>	Muss <code>ASSERT</code> sein.	Ja
<code>response</code>	Name der zuvor durch einen REST-EXEC gespeicherten Response.	Ja
<code>source</code>	Für <code>MATCH_REFERENCE</code> wird der JSON-Body mit <code>BODY</code> geprüft.	Ja
<code>action</code>	Muss <code>MATCH_REFERENCE</code> sein.	Ja
<code>path</code>	Wird bei <code>MATCH_REFERENCE</code> nicht benötigt .	Nein
<code>expected</code>	Wird bei <code>MATCH_REFERENCE</code> nicht benötigt .	Nein
<code>ignore</code>	JSON-Pfade dynamischer Felder, die vom Vergleich ausgeschlossen werden.	Nein
<code>unordered</code>	Regeln für Arrays, deren Reihenfolge	Nein

	beim Vergleich keine Rolle spielt.	
<code>unordered.path</code>	JSON-Pfad des Arrays.	Ja, wenn <code>unordered</code> verwendet wird
<code>unordered.matchBy</code>	JSON-Pfad eines stabilen Schlüssels innerhalb eines Array-Elements.	Nein, aber empfohlen

Kurzform: `MATCH_REFERENCE` ist für die Validierung einer **kompletten REST-Response** vorgesehen. Klassische ASSERTs mit `path`, `action` und `expected` bleiben die bessere Wahl, wenn lediglich einzelne konkrete Werte geprüft werden sollen.

2.3. BUFFER (Extraktion in Variablen)

Extrahiert einzelne Werte aus der gespeicherten HTTP-Antwort und legt sie in den Runtime-Variablen des Testfalls unter dem im Parameter `name` angegebenen Schlüssel ab.

- **Besonderheit der Syntax:** Im Gegensatz zur Assertion verwendet die Extraktion aus dem `BODY` die **Jackson-Pointer-Syntax**. Der Engine konvertiert das Standardformat (z. B. `$.data.user`) automatisch in das Pointer-Format (`/data/user`).
- **COUNT im Buffer:** Wenn `action: COUNT` übergeben wird, wird die Größe des gefundenen JSON-Arrays oder -Objekts in die Variable geschrieben.

3. Referenzübersicht der Parameter nach Operationen

Schritt (op)	Parameter	Beschreibung	Erforderlich
EXEC (<i>default</i>)	method	HTTP-Methode (GET , POST , PUT , DELETE ...)	Ja
	url	Zieladresse (unterstützt Platzhalter)	Ja
	endpoint	Basisadresse, die	Nein

		{{endpoint}} in der URL ersetzt	
	headers	Map der HTTP-Header, die mit der Anfrage gesendet werden	Nein
	body	Payload der Anfrage in Textform	Nein
	response	Name des Schlüssels, unter dem die Antwort im Buffer gespeichert wird	Nein (Empfohlen)
ASSERT	response	Schlüssel, unter dem die Antwort zuvor gespeichert wurde	Ja
	source	Quelle für die Validierung: STATUS , BODY oder HEADERS	Ja
	path	JsonPath-Ausdruck für BODY ili genauer Name des HTTP-Headers	Nein
	action	Typ der Überprüfung (EQUALS , CONTAINS ,	Ja

		EXISTS , COUNT ...)	
	expected	Wert, mit dem das tatsächliche Ergebnis verglichen wird	Ja
	value	Unterkriterium zum Filtern bei der COUNT - Aktion	Nein
BUFFER	response	Schlüssel, unter dem die Antwort zuvor gespeichert wurde	Ja
	source	Quelle zum Lesen: STATUS , BODY oder HEADERS	Ja
	path	Pfad zum Datenpunkt (automatische Jackson- Konvertierung)	Ja
	name	Name der Variable, in der der Wert dauerhaft gespeichert wird	Ja
	action	Auf COUNT setzen, wenn die Anzahl der Elemente gewünscht ist	Nein

Praxis-Tutorial: Kompletter REST-Lebenszyklus

Dieses Beispiel zeigt die Erstellung einer POST-Anfrage mit dem Abrufen eines dynamischen Tokens über die **BUFFER**-Operation, gefolgt vom Senden einer neuen Anfrage mit diesem Token und der abschließenden **ASSERT**-Verifizierung des Status und des Antwort-Bodys.

Komplettes YAML-Szenario

1 | YAML

```
1  testCases:
2    - id: "TC_REST_WORKFLOW_DEMO"
3      description: "Kombinierter REST-Test: Authentifizierung, Buffer und
4        variables:
5          api_host: "https://api.example.com"
6          search_term: "Chanda"
7        steps:
8          # 1. Schritt: Authentifizierung und Speichern der vollständigen A
9          - type: REST
10             method: "POST"
11             url: "{{endpoint}}/api/v1/auth"
12             endpoint: "{E[api_host]}"
13             headers:
14               Content-Type: "application/json"
15               X-Client-Type: "{NULL}" # Aus der Anfrage entfernt
16               X-Trace-Id: "{EMPTY}" # Als leerer String "" gelesen
17             body: '{"user": "admin", "pass": "secret"}'
18             response: "auth_res"
19
20          # 2. Schritt: Extraktion des Tokens aus "auth_res" in die Variabl
21          - type: REST
22            op: BUFFER
23            response: "auth_res"
24            source: "BODY"
25            path: "$.security.token"
26            name: "my_token"
27
28          # 3. Schritt: Senden der Geschäftsanfrage unter Verwendung des ex
29          - type: REST
30            method: "GET"
31            url: "{E[api_host]}/api/v1/users?search={B[search_term]}"
32            headers:
33              Authorization: "Bearer {B[my_token]}"
34              Content-Type: "application/json"
35            response: "users_data"
36
37          # 4. Schritt: Validierung des Statuscodes der Antwort "users_data
38          - type: REST
39            op: ASSERT
40            response: "users_data"
41            source: "STATUS"
42            action: "EQUALS"
43            expected: "200"
44
45          # 5. Schritt: Validierung, ob die Benutzerliste den gesuchten Beg
46          - type: REST
47            op: ASSERT
48            response: "users_data"
49            source: "BODY"
50            path: "$.users[*].name"
51            action: "CONTAINS"
52            expected: "{B[search_term]}"
```

Darstellung der Konsolenausgabe (Log-Output)

Wenn der Verbose-Modus (`isVerbose`) und das Drucken der Ausführung (`printExecution`) aktiviert sind, gibt der `RestResponsePrinter` in Zusammenarbeit mit dem Engine-Logger den detaillierten Ablauf im folgenden Format aus:

1 Plaintext

```

1  >>> CALL      : POST https://api.example.com/api/v1/auth
2  >>> HEADER     : Content-Type: application/json
3  >>> HEADER     : X-Trace-Id:
4  >>> BODY       : {"user": "admin", "pass": "secret"}
5
6  <<< HTTP       : 200
7  <<< BODY       : {"security":{"token":"eyJhbGciOi...", "status":"ACTIV
8
9  >>> BUFFER     : Value [eyJhbGciOi...] stored in variable [my_token]
10
11 >>> CALL      : GET https://api.example.com/api/v1/users?search=Chand
12 >>> HEADER     : Authorization: Bearer eyJhbGciOi...
13 >>> HEADER     : Content-Type: application/json
14
15 <<< HTTP       : 200
16 <<< BODY       : {"users":[{"id":1,"name":"Chanda"}, {"id":2,"name":"SI
17
18 >>> SUCCESS: EQUALS | Expected: [200], Actual: [200]
19 >>> SUCCESS: CONTAINS | Expected: [Chanda], Actual: [{"Chanda", "SI
20
```

Troubleshooting & Häufige Fehler

• Fehlschlag mit der Meldung `Path not found` während der `BUFFER`-Operation:

- *Ursache:* Senden eines falschen JsonPath-Ausdrucks. Da die `BUFFER`-Operation intern die Jackson-Bibliothek verwendet, konvertiert sie Punkte in Schrägstriche. Wenn Ihr Pfad komplexe Schlüssel mit Sonderzeichen enthält (z. B. `$.['metadata.info']`), schlägt die Konvertierung in den Jackson-Pointer fehl. Verlassen Sie sich bei komplexen Strukturen ausschließlich auf den `ASSERT`-Schritt, der den vollwertigen Jayway JsonPath verwendet.

• Fehler `Validation failed: Path [...] returned multiple elements`:

- *Ursache:* Sie haben versucht, die Aktion `EQUALS` oder `VERIFY` auf einen Pfad anzuwenden, der ein Wildcard enthält (z. B. `$.items[*].id`), welcher ein Array von Elementen statt eines einzelnen Wertes zurückgegeben hat.
- *Lösung:* Ändern Sie die Aktion in `CONTAINS` (Prüfung auf Existenz innerhalb dieses Arrays) oder targetieren Sie explizit einen Index (z. B. `$.items[0].id`).

• Zufällige Verbindungsabbrüche (`Connection reset` / `header parser received no bytes`):

- *Verhalten:* Der REST-Engine verfügt über einen integrierten Schutz für diese Situationen. Wenn Sie die Warnung `[RETRY] Attempt 1/3 failed...` in der Konsole bemerken,

bedeutet dies, dass der Engine das Netzwerkproblem erfolgreich abgefangen, einen neuen Socket geöffnet und die Ausführung des Tests fortgesetzt hat, ohne den gesamten Build abzubrechen.

- **Die Assertion gibt bei einem Fehlschlag nicht den Error-Body aus:**

- *Ursache:* Die Methode `logAssertionResult` hat den Parameter `body` beim Aufruf aus `handleAssertion` auf `null` gesetzt. Wenn Sie die Ausgabe des vollständigen Antwort-Bodys im Falle eines Fehlschlags der Assertion wünschen, muss im Java-Code `res.getBody()` anstelle des zweiten `null` innerhalb dieses Funktionsaufrufs übergeben werden.